

sa 변경분

필사해 둔 sa.py 에서 고칠 줄만 — 녹색 줄을 새 줄번호 자리에 써 넣으면 된다

fe6c42d → 2026-08-22 · 28220fe+수정중 | +14 / -3 줄 · 코드 3쪽 · 새로 쓸 줄 67-73, 79, 87, 92, 97, 163-165

바뀐 내용

· 버스트마다 판독 숫자줄을 찍는다: $x^2 \cdot x^4 \cdot x^8$ 판의 최대 바늘 주파수(f, Hz)와 돌출(d, dB), AM 검파 봉우리(am f, d) — 그림을 말로 옮기는 대신 이 줄(검출 코드 포함)을 받아치면 fc·심볼레이트·변조가 숫자로 전해진다. peak() 함수 하나가 새로 생기고, burst_row 가 숫자줄도 돌려주며, 호출부가 그 줄을 출력한다.

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 줄이 사라짐 · **ㄴ** = 윗줄에 붙는 이어짐. 고친 뒤 **python3 sa.py kat** 이 아래와 같아야 한다.

```
n 80000 fs 10000 길이 8.000s 포락선 분리(중앙 기준) 1.18 decades
find_bursts → 2개
  1 샘플 [ 11904, 27072) 1516.8 ms 전력 +4.0 dB ★판독
  2 샘플 [ 49984, 65024) 1504.0 ms 전력 +4.0 dB ★판독
sa b1 p2 f1100 d47 p4 f2200 d42 p8 f4400 d33 am f294 d39 #mmy9
sa b2 p2 f1123 d31 p4 f2200 d40 p8 f4400 d31 am f294 d38 #0fj5
sa kat n80000 f10000 s8.0 fb2 ev118 #7n3t
sa-kat.png 저장 — 위: 스펙트로그램+검출 띠+버스트 번호, 아래: ★버스트별 [PSD | $x^2$  | $x^4$  | $x^8$  |AM] ( $x^2$  바늘=BPSK,  $x^4$ =QPSK,  $x^8$ =8PSK, AM 봉우리=심볼
```

```

1 1 """sa.py - 종합 관찰 프로브 (장비 필사용, signus/ 옆에 저장). 한 번에:
2 2 디버그 출력(포락선·버스트 표) + <캡처>.sa.png (스펙트로그램 + 검출 때 + 버스트 번호 +
3 3 상위 버스트별 [PSD | x^2 | x^4 | x^8 | AM검파] 판독 행 - 바늘이 처음 서는 판이 변조).
4 4 python3 sa.py kat # 자기검증: 표지 기대 출과 비교, sa-kat.png 생성
5 5 python3 sa.py <캡처파일> [K] # 판독할 버스트 수 K (기본 4, 에너지 상위부터)
6 6 x^m 판은 접힌 축(0~fs/2, 음수 주파수를 겹쳐 최대값) - 바늘 위치가 m·fc (mod fs) 다.
7 7 """
8 8 import struct
9 9 import sys
10 10 import zlib
11 11
12 12 import numpy as np
13 13 from scipy.signal import stft, welch
14 14
15 15 from signus import dsp, sigio
16 16 from signus.cli import check_code
17 17
18 18 GLYPH = {"0": (14, 17, 19, 21, 25, 17, 14), "1": (4, 12, 4, 4, 4, 4, 14),
19 19 "2": (14, 17, 1, 2, 4, 8, 31), "3": (31, 2, 4, 2, 1, 17, 14),
20 20 "4": (2, 6, 10, 18, 31, 2, 2), "5": (31, 16, 30, 1, 1, 17, 14),
21 21 "6": (6, 8, 16, 30, 17, 17, 14), "7": (31, 1, 2, 4, 8, 8, 8),
22 22 "8": (14, 17, 17, 14, 17, 17, 14), "9": (14, 17, 17, 15, 1, 2, 12)}
23 23
24 24
25 25 def draw_text(img, row, col, text, s=2):
26 26     for ch in text:
27 27         for r, bits in enumerate(GLYPH[ch]):
28 28             for c in range(5):
29 29                 if bits >> (4 - c) & 1:
30 30                     img[row + r * s:row + r * s + s, col + c * s:col + c * s + s] = 1.0
31 31                 col += 6 * s
32 32
33 33
34 34 def png_gray(img, path):
35 35     img = np.clip(img * 255, 0, 255).astype(np.uint8)
36 36     h, w = img.shape
37 37
38 38     def chunk(tag, data):
39 39         c = tag + data
40 40         return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))
41 41
42 42     raw = b"".join(b"\x00" + img[r].tobytes() for r in range(h))
43 43     with open(path, "wb") as fh:
44 44         fh.write(b"\x89PNG\r\n\x1a\n"
45 45             + chunk(b"IHDR", struct.pack(">IIBBBBB", w, h, 8, 0, 0, 0, 0))
46 46             + chunk(b>IDAT", zlib.compress(raw, 6)) + chunk(b"IEND", b""))
47 47
48 48
49 49 def panel(fvals, dbvals, wp=500, hp=120, grid=()):
50 50     """스펙트럼 곡선 한 판 - 바늘 보존을 위해 표시 칸마다 최대값 풀링."""
51 51     cols = np.clip((fvals / fvals.max() * (wp - 1)).astype(int), 0, wp - 1)
52 52     cur = np.full(wp, -1e9)
53 53     np.maximum.at(cur, cols, dbvals)
54 54     bad = cur < -1e8
55 55     cur[bad] = np.interp(np.flatnonzero(bad), np.flatnonzero(~bad), cur[~bad])
56 56     g_lo, g_hi = np.percentile(cur, 5) - 2, cur.max() + 2
57 57     img = np.zeros((hp, wp))
58 58     for gf in grid:
59 59         img[:,6, int(gf / fvals.max() * (wp - 1))] = 0.35
60 60     rows = ((hp - 1) * (1 - np.clip((cur - g_lo) / (g_hi - g_lo), 0, 1))).astype(int)

```

```

61 61     for cx in range(wp):
62 62         r0, r1 = (rows[cx], rows[cx - 1]) if cx else (rows[0], rows[0])
63 63         img[min(r0, r1):max(r0, r1) + 1, cx] = 1.0
64 64     return img
65 65
66 66
67 + def peak(fv, dbv, fmin=20.0):
68 +     """판 하나의 최대 바늘: 주파수(Hz)와 중앙값 대비 돌출(dB) -- 숫자로 받아칠 수 있게."""
69 +     ok = fv >= fmin
70 +     i = int(np.argmax(dbv[ok]))
71 +     return f"f{round(float(fv[ok][i]))} d{round(float(dbv[ok][i] - np.median(dbv[ok])))}"
72 +
73 +
67 74 def burst_row(z, fs, label):
68 75     """한 버스트의 판독 행: [PSD | x^2 | x^4 | x^8 | AM]. x^m 은 음수 주파수를 접는다."""
69 76     grid = tuple(fs / 2 * k / 5 for k in (1, 2, 3, 4))
70 77     fr, praw = welch(z.real, fs=fs, nperseg=min(2048, z.size))
71 78     ps = [panel(fr, 10 * np.log10(praw + 1e-18), grid=grid)]
79 +     nfft, nums = 1 << 15, []
73 80     f = np.fft.fftfreq(nfft, 1 / fs)
74 81     half = nfft // 2
75 82     for m in (2, 4, 8):
76 83         mag = np.abs(np.fft.fft((z ** m) * np.hanning(z.size), nfft))
77 84         fold = np.maximum(mag[:half], np.r_[mag[0], mag[1:half][::-1] * 0
78 85             + mag[half + 1:][::-1]])
79 86         ps.append(panel(f[:half], 20 * np.log10(fold + 1e-12), grid=grid))
87 +         nums.append(f"p{m} " + peak(f[:half], 20 * np.log10(fold + 1e-12)))
80 88     u = np.abs(z) ** 2
81 89     u = u - u.mean()
82 90     su = np.abs(np.fft.rfft(u * np.hanning(u.size), nfft))
83 91     ps.append(panel(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12), grid=grid))
92 +     nums.append("am " + peak(np.fft.rfftfreq(nfft, 1 / fs), 20 * np.log10(su + 1e-12)))
84 93     div = np.full((120, 4), 0.5)
85 94     row = np.hstack(sum([p, div] for p in ps[:-1]), []) + [ps[-1]]
86 95     tag = np.zeros((120, 26))
87 96     draw_text(tag, 4, 4, label)
97 +     return np.hstack([tag, row]), " ".join(nums)
89 98
90 99
91 100 if len(sys.argv) < 2:
92 101     raise SystemExit("사용법: python3 sa.py kat | python3 sa.py <캡처파일> [버스트수 K]")
93 102 arg = sys.argv[1]
94 103 kmax = int(sys.argv[2]) if len(sys.argv) > 2 else 4
95 104 if arg == "kat":
96 105     fs, n = 10000.0, 80000
97 106     rng = np.random.default_rng(21) # 시드 고정 -- numpy 가 재현을 보장한다
98 107     x0 = 0.18 * rng.standard_normal(n)
99 108     t = np.arange(15000) / fs
100 109     for s0, fc, ph in ((12000, 550.0, 2), (50000, 550.0, 4)): # 앞=BPSK, 뒤=QPSK
101 110         up = np.zeros(15000, complex)
102 111         up[:34] = np.exp(2j * np.pi * rng.integers(0, ph, 442) / ph)
103 112         sym = np.convolve(up, np.hanning(68), "same") # 간이 펄스 성형
104 113         sym /= np.sqrt(np.mean(np.abs(sym) ** 2))
105 114         x0[s0:s0 + 15000] += (sym * np.exp(2j * np.pi * fc * t)).real * np.sqrt(2)
106 115         name = "sa-kat"
107 116     else:
108 117         x0, meta = sigio.read(arg)
109 118         fs = meta.fs
110 119         name = arg + ".sa"
111 120     xa = dsp.analytic(x0)

```

```

112 121 xa = xa - xa.mean()
113 122 n = xa.size
114 123 fb = dsp.find_bursts(xa, fs)
115 124 full = fb == [(0, n)]
116 125 pw = np.abs(xa) ** 2
117 126 lp = np.log10(np.maximum(np.convolve(pw, np.ones(64) / 64, "same"), 0) + 1e-20)
118 127 et = np.percentile(lp, 50)
119 128 ev = float(lp[lp >= et].mean() - lp[lp < et].mean())
120 129 print(f"n {n} fs {fs:.0f} 길이 {n / fs:.3f}s 포락선 분리(중앙 기준) {ev:.2f} decades")
121 130 print(f"find_bursts → {'통짜 [(0,n)] = 미검출' if full else str(len(fb)) + '개'}")
122 131 order = sorted(range(len(fb)), key=lambda i: -float(pw[fb[i][0]:fb[i][1]].sum()))
123 132 pick = sorted(order[:kmax]) if not full else []
124 133 for i, (s0, e0) in enumerate(fb if not full else []):
125 134     star = "★판독" if i in pick else ""
126 135     print(f" {i + 1:>2} 샘플 [{s0:>8}, {e0:>8}] {1000 * (e0 - s0) / fs:8.1f} ms"
127 136           f" 전력 {10 * np.log10(pw[s0:e0].mean() / (pw.mean() + 1e-30) + 1e-30):+5.1f} dB{star}
128 137           ")
129 138 x_disp = x0.real if np.iscomplexobj(x0) else x0
130 139 _, _, z = stft(x_disp, fs=fs, window="hamming", nperseg=256, noverlap=128,
131 140           return_onesided=True, boundary=None, padded=False)
132 141 db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
133 142 lo = float(np.percentile(db, 25))
134 143 rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
135 144 spec = np.clip((db - lo) / rg, 0, 1)[::-1]
136 145 kp = max(1, spec.shape[1] // 4000)
137 146 spec = spec[:, :spec.shape[1] // kp * kp].reshape(spec.shape[0], -1, kp).max(2)
138 147 pw_row = 26 + 5 * 500 + 4 * 4 # 판독 행 폭 -- 스트립을 여기에 맞춰 늘린다
139 148 fx = max(1, pw_row // spec.shape[1])
140 149 spec = np.repeat(spec, fx, axis=1)
141 150 ncol = spec.shape[1]
142 151 mark = np.zeros((18, ncol))
143 152 lane = np.zeros(ncol)
144 153 for i, (s0, e0) in enumerate(fb if not full else []):
145 154     c0 = (s0 // 128) // kp * fx
146 155     c1 = min(ncol - 1, (e0 // 128) // kp * fx)
147 156     lane[c0:c1 + 1] = 1.0
148 157     draw_text(mark, 2, min(c0 + 2, ncol - 14 * len(str(i + 1))), str(i + 1))
149 158 rows = [mark, spec, np.full((2, ncol), 0.35), np.tile(lane, (10, 1)),
150 159         np.full((6, ncol), 0.0)]
151 160 width = max(ncol, pw_row)
152 161 rows = [np.pad(r, ((0, 0), (0, width - r.shape[1]))) for r in rows]
153 162 for i in pick:
163 +     r, num = burst_row(xa[fb[i][0]:fb[i][1]], fs, str(i + 1))
164 +     ln = f"sa b{i + 1} {num}" # 바늘 주파수·돌출 -- 받아칠 판독 숫자줄
165 +     print(f"{ln} #{check_code(ln)}")
155 166     rows += [np.pad(r, ((0, 0), (0, width - r.shape[1])), np.full((6, width), 0.0)]
156 167 png_gray(np.vstack(rows), name + ".png")
157 168 line = (f"sa {'kat' if arg == 'kat' else 'cap'} n{n} f{fs:.0f} s{n / fs:.1f}"
158 169         f" fb{0 if full else len(fb)} ev{round(100 * ev)}")
159 170 print(f"{line} #{check_code(line)}")
160 171 print(f"{name}.png 저장 - 위: 스펙트로그램+검출 띠+버스트 번호, 아래: ★버스트별"
161 172       " [PSD|x2|x4|x8|AM] (x2 바늘=BPSK, x4=QPSK, x8=8PSK, AM 봉우리=심볼레이트)")

```